US Market Advisory RAG System

A Retrieval-Augmented Generation Architecture
for Grounded Financial Market Intelligence

Syed Aqdas Munir
B24F1373AI030 | Section: AI-BLUE
AI & Financial Systems | Project Report | April 2026

Abstract

Large language models (LLMs) applied to financial question answering suffer from three well-documented failure modes: hallucinated figures, stale parametric knowledge, and unattributed claims that cannot be audited in regulated settings. This report presents the design and architecture of the ***US Market Advisory RAG System****, a production-grade Retrieval-Augmented Generation (RAG) platform that addresses all three limitations by grounding every generated response in retrieved, timestamped, and source-attributed documents. The system ingests authoritative data across three lanes — US equities and ETF data, macroeconomic indicators from FRED, BEA, and BLS, and regulatory filings from SEC EDGAR and the Federal Register — and indexes them as dense vector embeddings using OpenAI* `text-embedding-3-large` *(3072 dimensions) in a Qdrant vector store with metadata-filtered approximate nearest-neighbour search. At query time, a FastAPI service orchestrates retrieval, optional cross-encoder reranking, prompt construction, GPT-4o generation, and structured citation formatting, returning answers with inline source references. Domain guardrails enforce scope boundaries and financial advice disclaimers. The system is containerised with Docker Compose, ships a CI/CD pipeline via GitHub Actions, and stores full query and ingestion audit trails in PostgreSQL. Evaluation covers Recall@k, MRR, faithfulness scoring, and guardrail classification accuracy.*

Keywords: Retrieval-Augmented Generation, RAG, Financial NLP, Vector Search, Qdrant, OpenAI Embeddings, GPT-4o, FastAPI, Financial Market Intelligence, Citation Grounding

I. Introduction

A. Motivation

Financial professionals require answers that are accurate, current, and attributable. General-purpose LLMs fail all three criteria at scale: they fabricate ticker prices, miss regulatory amendments issued after their training cutoff, and produce responses with no verifiable source trail.

Retrieval-Augmented Generation [1] directly addresses these gaps by decoupling the knowledge base from the model weights. The model is constrained to generate only from retrieved context, and every factual claim can be traced to a specific source document and timestamp.

B. Problem Statement

Three failure modes are observed when deploying vanilla LLMs in financial contexts:

1. **Hallucinated figures:** Incorrect stock prices, GDP values, and regulation citations produced from parametric memory.
2. **Stale knowledge:** Training cutoffs leave models unaware of recent SEC rule amendments, earnings events, or FOMC decisions.
3. **Unattributable claims:** No source trail — a critical compliance gap in regulated financial environments.

C. Contributions

This work presents:

- A three-lane ingestion architecture covering equities, macroeconomics, and financial regulation with lane-aware metadata filtering at retrieval time.
- A full RAG pipeline integrating dense vector retrieval, cross-encoder reranking, structured prompt construction, GPT-4o generation, and citation post-processing.
- Domain guardrails for scope enforcement and financial advice framing.
- A production-ready FastAPI service with PostgreSQL audit logging, Alembic migrations, containerised deployment, and GitHub Actions CI/CD.

II. Related Work

RAG was formally introduced by Lewis et al. [1] as a method for knowledge-intensive NLP tasks.

Subsequent work [2, 3] demonstrated that retrieval-conditioned generation substantially reduces hallucination rates compared to closed-book generation. In the financial domain, BloombergGPT [4] established the value of domain-specific pretraining, though at significant compute cost. This system takes an alternative approach: using a general-purpose frontier model (GPT-4o) conditioned on retrieved specialist documents, avoiding costly pretraining while maintaining up-to-date knowledge through continuous ingestion.

Hybrid retrieval methods combining dense and sparse signals [5] are employed optionally for regulatory queries where keyword precision matters. Cross-encoder reranking [6] is incorporated as an optional second-stage filter to improve precision before prompting.

III. System Architecture

A. Architectural Layers

The system is organized into five layers interacting linearly at query time and in parallel fan-out at ingestion time:

Layer 1 — Data Ingestion: Source connectors, loaders, preprocessing, metadata enrichment.
Layer 2 — Vector Store: Qdrant collection, dense embeddings, ANN search.
Layer 3 — RAG Pipeline: Retrieve → rerank → prompt → generate → cite.
Layer 4 — API Service: FastAPI, auth, validation, structured logging.
Layer 5 — Database: PostgreSQL, document registry, query audit, run history.

B. Query-Time Data Flow

At inference time the pipeline executes as:

$$Q \xrightarrow{\text{guard}} \mathcal{R}(Q) \xrightarrow{\text{rerank}} \mathcal{P}(Q, \mathcal{R}) \xrightarrow{\text{LLM}} A \xrightarrow{\text{cite}} R$$

where Q is the user query, $\mathcal{R}(Q)$ the retrieved chunk set, $\mathcal{P}(Q, \mathcal{R})$ the assembled prompt, A the raw LLM output, and R the final structured response with citations.

IV. Data Sources & Ingestion

A. Lane A — US Equities

Daily OHLCV prices for S&P 500 constituents, Nasdaq-100, and major sector ETFs are pulled from Alpha Vantage and iShares CSV exports. Corporate action announcements and earnings calendars are included. Data is parsed by `csv_loader.py` and normalised to a common time-series schema before chunking.

B. Lane B — Macroeconomic Indicators

The St. Louis FRED API provides GDP (real and nominal), CPI, PCE deflator, federal funds rate, M2 money supply, and 10-year treasury yield series. BEA and BLS flat files add non-farm payroll and unemployment data. All series are normalised to a unified {`series_id, date, value, unit`} schema and ingested on the BLS/BEA release schedule.

C. Lane C — Financial Regulation

SEC EDGAR full-text search and the Federal Register API supply final rules, proposed rules, no-action letters, and regulatory notices. `pdf_loader.py` and `web_loader.py` extract text; `clean_text.py` removes headers, footers, and boilerplate. A change-detection hash prevents re-ingestion of unmodified documents.

D. Ingestion Pipeline

The seven-step pipeline executed by `scripts/ingest.py` is:

S1. Download — Pull raw assets; write to `data/raw/` with a fetch manifest.
S2. Load — Lane-specific loaders produce normalised intermediate documents.
S3. Clean — Strip noise, OCR artifacts, repeated headers.
S4. Normalise — Standardise tables and numeric series.
S5. Chunk — Structural chunking for regulatory PDFs; 512-token sliding window (64-token overlap) for narrative content.
S6. Embed — Batch embed chunks via `text-embedding-3-large`; 128 texts per API call.
S7. Store — Upsert chunk records to PostgreSQL; upsert vectors and payloads to Qdrant in batches of 256.

E. Chunk Metadata Schema

Every chunk payload carries the fields listed in Table 1.

Table 1: Per-chunk metadata fields stored in Qdrant payload and PostgreSQL.

Field	Type	Purpose
`chunk_id`	UUID	Unique chunk identifier
`doc_id`	UUID	Parent document
`lane`	Enum	equities / macro / regulation
`source`	String	FRED, SEC, BLS, etc.
`section`	String	Series or document section
`published_at`	Timestamp	Data publication date
`entity_tags`	List	Tickers, agencies, series IDs
`file_type`	Enum	pdf / csv / html / json

V. Embedding & Vector Store

A. Embedding Model

OpenAI `text-embedding-3-large` produces 3072-dimensional vectors with an 8192-token context window. The model achieves state-of-the-art MTEB retrieval scores and supports Matryoshka representation learning [7], allowing dimensionality truncation to 1536 or 256 for cost-performance trade-offs without retrain-

ing. At approximately $0.13 per million tokens it is economical for the ingestion volumes expected at this scale. Both document and query embeddings use the same model and normalisation, ensuring cosine similarity is directly comparable.

B. Qdrant Configuration

Qdrant is selected for its native payload filtering, which applies metadata constraints at the HNSW graph traversal level rather than as a post-retrieval scan, and its optional hybrid search combining dense vectors with sparse BM25-style signals. The collection is configured with cosine distance, 3072-dimensional vectors, and HNSW parameters $m = 16$, $ef_construct = 100$.

C. Two-Stage Retrieval

Stage 1 — Dense ANN: The query embedding is searched against the Qdrant collection returning top-k candidates (default $k = 20$) after applying any requested metadata filters (lane, date range, entity tags).

Stage 2 — Cross-Encoder Reranking (optional): `reranker.py` applies `cross-encoder/ms-marco-MiniLM-L-6-v2` to rescore the top-20 candidates, returning the final top-n chunks (default $n = 6$) at higher precision. This stage is toggled via `RERANKING_ENABLED`.

VI. RAG Pipeline

A. Guardrails

Two guardrail checks execute before retrieval.

Scope check: Queries are classified against an in-scope whitelist covering US equities, macroeconomic indicators, and US financial regulations. Out-of-scope queries return a structured refusal without entering the retrieval pipeline, preventing unnecessary API calls and controlling output scope.

Financial advice framing: Any generated response containing forward-looking statements, price targets, or investment recommendations is automatically appended with a regulatory disclaimer indicating the response is informational and does not constitute investment advice.

B. Prompt Construction

`prompt.py` uses a Jinja2 template to build the final prompt with four components: (i) a system instruction defining role, scope, and citation behaviour; (ii) retrieved context chunks labelled with `[SOURCE_N]` tokens tied to their metadata; (iii) the user query; and (iv) an instruction to answer only from the provided context and to cite sources inline using the `[SOURCE_N]` tokens. The template enforces citation anchoring, preventing the model from generating unattributed factual claims.

C. LLM Generation

`app/llm/client.py` wraps the OpenAI Chat Completions API using GPT-4o with temperature 0.2 for factual responses, a 1024-token generation budget, and streaming support for low-latency downstream clients. Three-attempt exponential backoff handles `RateLimitError` and `APIError` transparently.

D. Citation Post-Processing

`citations.py` scans the raw generation output, extracts all referenced `[SOURCE_N]` tokens, and resolves each to its full chunk record (doc_id, source, section, published_at). The response payload returns a structured `citations` array alongside the answer text, enabling clients to render footnotes, hyperlinks, or inline source badges.

VII. API Design

A. Authentication

All endpoints except `GET /health` require an `X-API-Key` header validated against a bcrypt-hashed store in environment configuration. A JWT flow is available via `core/security.py` for per-user audit trails in multi-tenant deployments.

B. Endpoint Contract

`GET /health` returns service liveness plus dependency status for PostgreSQL, Qdrant, and OpenAI reachability.

`POST /v1/query` is the primary RAG endpoint. The request carries the natural language query, `top_k` ($1 \leq k \leq 20$), an optional `time_range` with ISO 8601 bounds, optional lane filters, and a boolean `include_citations` flag. The response returns the grounded answer string, a structured `citations` array, a `query_id` UUID for audit lookup, and end-to-end `latency_ms`.

`POST /v1/ingest` is a protected endpoint triggering an ingestion run for specified lanes with optional date bounds and a `force_reingest` flag. It returns a run ID and a status polling URL.

C. Validation

All request and response schemas are defined in Pydantic v2 (`api/schemas.py`). Validation failures return HTTP 422 with field-level error messages. Query text is truncated at 2000 characters and stripped of control characters before any downstream use.

VIII. Database Design

The PostgreSQL schema is managed with Alembic migrations and consists of four core tables (Table 2).

Table 2: PostgreSQL schema. All primary keys are UUID.

Table	Purpose
`documents`	Source registry: URL, lane, content hash, timestamps
`chunks`	Chunk text, position index, token count, FK to document
`query_logs`	Full query audit: text, filters, latency, answer hash
`ingestion_runs`	Lane, document count, error count, start/finish time

The `create_all()` startup pattern is replaced by a migration check: on startup the service compares the Alembic revision head with the live database state and exits with an error if a pending migration is detected, preventing silent schema drift in production.

IX. Deployment

A. Containerisation

`docker-compose.yml` defines four services: the FastAPI application (Python 3.11 slim), PostgreSQL 16, Qdrant, and an optional Redis cache for query deduplication. All credentials are injected via environment variables; `.env.example` documents every required key without storing values. The `make setup` target runs `bootstrap.sh` to initialise the environment and apply pending Alembic migrations in one command.

B. CI/CD

`ci.yml` runs on every push: `ruff` linting, `mypy` type checks, `pytest` with in-memory Qdrant and SQLite fixtures, and a minimum 80% line coverage gate. `cd.yml` triggers on version tags: Docker image build, push to a container registry, and optional deployment to DigitalOcean App Platform via `doctl`.

X. Evaluation

A. Retrieval Metrics

`scripts/eval_rag.py` runs against a curated set of (query, ground-truth chunk) pairs for each lane. Metrics computed include Recall@k for $k \in \{1, 5, 10\}$, Mean Reciprocal Rank (MRR), and Precision@n after reranking. These are tracked per lane to detect degradation in individual data sources independently of overall system performance.

B. Generation Quality

Response faithfulness is scored using an LLM-as-judge prompt (or Ragas) that checks whether every factual claim in the generated answer is directly supported by a retrieved chunk. Answer relevancy is measured as the cosine similarity between the answer embedding and the query embedding. Citation coverage measures the fraction of factual claims backed by an inline `[SOURCE_N]` reference.

C. Guardrail Accuracy

A held-out test set of in-scope and out-of-scope queries measures refusal precision (fraction of out-of-scope queries correctly refused) and pass-through recall (fraction of in-scope queries correctly admitted to the pipeline), providing a balanced view of guardrail calibration.

XI. Security

API keys are stored as bcrypt hashes; plaintext keys never persist server-side. Rate limiting is applied per key via `slowapi` middleware (default 60 requests/minute). User query strings are truncated and sanitised before embedding or prompt injection. The system prompt includes explicit instructions to disregard attempts to override scope, ignore source constraints, or solicit actions outside the QA role, mitigating prompt injection attacks. All secrets are managed through environment variables with no hardcoded credentials anywhere in the codebase.

XII. Technology Stack

Table 3 summarises the full technology stack with rationale.

Table 3: Technology stack and design rationale.

Component	Technology & Rationale
API Framework	**FastAPI 0.111+** — async, typed, auto-generated OpenAPI docs
Embedding	**text-embedding-3-large** — SOTA MTEB retrieval, 3072-dim
LLM	**GPT-4o** — best factual accuracy on financial text
Vector Store	**Qdrant** — payload filtering, hybrid search, Docker-native
Relational DB	**PostgreSQL 16** — audit log, metadata, ACID guarantees
ORM/Migrations	**SQLAlchemy 2 + Alembic** — async ORM, schema versioning
Validation	**Pydantic v2** — fast, typed request/response models
Reranker	**MiniLM-L6 cross-encoder** — local, lightweight, no API cost
Dependency Mgmt	**Poetry** — deterministic lockfile, isolated virtualenv
Containers	**Docker + Compose** — dev/prod parity, one-command setup
CI/CD	**GitHub Actions** — lint, test, build, deploy automation
Logging	**structlog (JSON)** — correlation-ID aware, machine-parseable

XIII. Conclusion

This report has presented the complete architecture of the US Market Advisory RAG System: a production-grade platform for citation-backed financial intelligence. By combining three-lane authoritative ingestion, high-dimensional dense retrieval with metadata-filtered vector search, cross-encoder reranking, and GPT-4o generation conditioned on retrieved context, the system resolves the core failure modes of vanilla

LLMs in financial settings — hallucination, staleness, and unattributable output.

The modular design allows individual components (embedding model, vector store, LLM backend, data lanes) to be upgraded independently without architectural changes. The evaluation framework provides continuous visibility into retrieval quality, generation faithfulness, and guardrail accuracy. When fully deployed, the system provides financial professionals with a trustworthy, auditable, scope-controlled AI assistant that answers only what it can verify and cites every claim it makes.

References

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] K. Guu *et al.*, "REALM: Retrieval-Augmented Language Model Pre-Training," *Proceedings of ICML*, 2020.

[3] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," *Proceedings of EACL*, 2021.

[4] S. Wu *et al.*, "BloombergGPT: A Large Language Model for Finance," *arXiv preprint arXiv:2303.17564*, 2023.

[5] T. Formal *et al.*, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," *Proceedings of SIGIR*, 2021.

[6] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[7] A. Kusupati *et al.*, "Matryoshka Representation Learning," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.